A history you can go back to

git for researchers, part 1: what a commit actually is

Florian Gärtner

Where we are today

	What Day 1 taught
	What Claude needs in every session
	What Claude needs only sometimes
	Your own skill
	Lunch
→	**A history you can go back to** ← you are here
	Where does this file live?
	Commit first, then let Claude move things. Then GitHub
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Describe** what git is and how it keeps the history of your project.
2. **Explain** what a commit contains, and why a piece of history cannot change without you noticing.
3. **Restore** any earlier state of your project by asking Claude, and get back out again.
4. **Read** the state and the history in what Claude ran, to tell where your project stands.

The folder you already have

This is a version control system

```
analysis.R
analysis_v2.R
analysis_v2_neu.R
analysis_final.R
analysis_final_REALLY.R
analysis_final_REALLY_2_Kommentare_Julia.R
```

It works. Sort of.

What it cannot tell you

- Which one produced the table in the submitted draft?
- What exactly is different between `final` and `final_REALLY`?
- It worked on Tuesday. What did I change?
- Julia sent comments on *which* version?

These are not filing problems. They are **provenance** problems.

The one that actually hurts

You change three files, the results change, and they are **worse**.

You cannot get back.

> **The promise of this block**
>
> Any earlier state of your project, back with one request. Including the state five minutes ago that you did not know you would want.

From your folder to any earlier state

Your folder
the files you work on
as they are now

→

A commit
the folder's state
a snapshot, with a message

→

The history
every commit, in order
with date and author

→

Undo
any earlier state
back with one request

Every commit stays, so any earlier state can come back.

What git is

Git keeps the history of a folder

Git is a **version control system**, a program on your computer that keeps the history of one project folder.

- When asked, git records the state of the folder. That record is a **commit**.
- Every commit stays, with its date, its author and a message.
- Any of them can be brought back.

A folder whose history git keeps is a **repository**. **GitHub** is something else: a website that keeps copies of repositories.

A snapshot, not a change

Does a commit store *what changed*?

No. It stores **the whole project, as it was at that moment**.

Every commit is a complete, independently readable state of your work. Nothing is reconstructed by replaying a chain.

> ⚠ **The mistake that makes git confusing**
>
> Thinking in diffs. If you believe history is a stack of changes, half of what git does looks arbitrary. If you believe it is a shelf of snapshots, it stops being mysterious.

The name is the content

Each snapshot gets a name computed **from its own contents**:

Name (the hash)	Message, written by Claude when committing
`d31400c`	Summarise after loading
`1d857ab`	Add the cleaning script
`a7fe9a1`	Start the project

Change one character anywhere in the project, and the name would have been different. So a commit cannot be quietly edited: altering the content alters the name, and every later snapshot points at the old one.

> **Why a researcher should care**
>
> This is what makes history **evidence** rather than a claim. “Table 3 came from commit `1d857ab`” is checkable by someone who does not trust you.

Three places, not one

Where	What it holds
Working directory	your files, right now, as any program sees them
Staging area	what is selected to go into the *next* snapshot
History	every snapshot taken so far

The staging area is the part with no equivalent in normal computer use, and the part worth understanding: it is how Claude commits *this* fix without the half-finished thing next to it, when you ask for the fix alone.

Claude does it, you read it

Making a repository

One request does it:

> Make this folder a git repository.

Claude runs:

```
1  git init -b main
```

That is the whole setup. A repository is a folder with a `.git` subfolder in it. Nothing left your machine, and nothing was registered anywhere.

Before the first commit, check what is in the folder. Data, private or licensed files, and keys stay out. A file you commit stays in the history even after you delete it, and a push later takes the whole history along. Your kit's rules tell Claude the same; check anyway.

Three commits

Three requests, one commit each:

> Create README.md with the line "# Analysis", and commit it.

> Write 01_clean.R: read data/raw.csv and drop the rows with missing values. Commit it.

> In 01_clean.R, print a summary right after loading. Commit it.

Each time, Claude runs `git add` and `git commit`, and writes the message. Read the message: it is for whoever has to find this commit in a year, which is you.

On Windows, `LF will be replaced by CRLF` may show in what Claude ran. It is about line endings, not an error.

Where am I?

One question answers it:

> What is the state of the repository?

Claude runs `git status`.

It reports what changed, what is staged, and what is not yet tracked: the answer to almost every “what is happening” moment. On a clean repository it says, in effect: nothing to do.

Reading the history

One request shows it:

> Show me the history.

```
git log --oneline
```

```
d31400c Summarise after loading
1d857ab Add the cleaning script
a7fe9a1 Start the project
```

Three snapshots, newest first. Each name is a place you can go back to.

Undo

One file, back to how it was

Something wrecked a script. First, what changed:

> What changed since the last commit?

```
1  M 01_clean.R
```

One request puts it back:

> Put 01_clean.R back as it was at the last commit.

```
1 git restore 01_clean.R
```

The file is back, and the state no longer shows the `M` for modified.

Read `git restore` as

"Take the version in the last snapshot and put it back in my folder."

Look at an earlier day, then come back

One request takes you there:

> Switch the folder to the first commit, so I can look at the files as they were.

```
1  git switch --detach a7fe9a1
```

The folder now holds `README.md` and nothing else. Coming back is one request too:

> Bring the project back to the newest commit.

```
1  git switch main
```

Both scripts are back, and the history still lists all three commits.

⚠ Look only, then come back

At an old snapshot (Claude may say “detached HEAD”), a commit made there stays out of your history. Come back before you change anything.

Make an earlier day the current one

One request does it:

> Put the whole project back as it was at the first commit, as a new commit. Keep the later commits in the history.

```
d9fdef4 Back to the state of the first commit
d31400c Summarise after loading
1d857ab Add the cleaning script
a7fe9a1 Start the project
```

The later snapshots are still there, so this undo can be undone too. Uncommitted work is overwritten: commit first.

⚠ The sharp alternative

`git reset --hard` also goes back, but takes the later commits off the history. Say “as a new commit” and “keep the later commits”, and Claude has no reason to reach for it; the kit’s `settings.json` denies it anyway.

The safety net under the safety net

One request shows it:

> Show me where the project has been, step by step.

```
git reflog
```

```
d9fdef4 HEAD@{0}: commit: Back to the state of the first commit
d31400c HEAD@{1}: checkout: moving from a7fe9a1… to main
a7fe9a1 HEAD@{2}: checkout: moving from main to a7fe9a1
d31400c HEAD@{3}: commit: Summarise after loading
```

Git records where you have *been*, not just what you committed. If a commit seems lost, ask Claude to look for it there.

The practical upshot

Once something is committed, it is very hard to lose. Almost every “I destroyed my repository” story is about work that was never committed.

Why this matters here

Provenance, concretely

- Which commit produced the table in the submission?
- What did the code look like when the referee report arrived?
- Did the result change before or after we fixed the merge?

All three are questions for the history. None of them are answerable from a folder of `_final_REALLY` files.

The history is Claude’s memory too

Every commit is a snapshot with a message, and Claude can read all of them. In a new session, ask:

> What did we change in the cleaning script last week, and why?

Claude answers from the commit history, **if each message says why**.

Rule of thumb

Commit often. Claude writes the message; check that it says why.

Undo is what makes an agent affordable

An assistant proposes changes far faster than you can review them.

If going back is one request, you can accept a suggestion, run it, look at the result, and throw it away. If going back means remembering what six files looked like, you will hesitate, and hesitating is expensive in a different way.

> **The working practice**
>
> Commit before you let Claude work. Then the question is never “can I undo this?”

Time for your project’s history

Exercise

Back in your project’s session (`/cd` and its path). **A menu: use what helps your project now. If it does not need any of it, use the time as you see fit.**

- **Your project is a git repository already?** Read its history with Claude. Ask what changed since the last commit, put a file back, look at an earlier commit and come back.
- **Not yet?** Block 2_6 makes it one, after 2_5 has shown what to keep out. For now, ask Claude which of your files you would most want back as they were last week.

What to take away

1. Git keeps the history of a project folder. Each commit is a snapshot of the whole project, with a message.
2. A commit’s name is computed from its content, so the history is evidence rather than a claim.
3. Any earlier state of your project, back with one request: one file, a look at an old snapshot, or the whole project as a new commit.
4. Commit before you let Claude work, and before an undo. What gets lost is work that was never committed.
5. The history is Claude’s memory too, if each message says why.

Not in this block

Branches, merging, worktrees. A branch is a second line of work in the same repository. You need one once two sessions work in the same project at the same time.

Where your work goes when your laptop dies. Remotes, and what to keep out of the repository.

> ⚠ **Everything so far is on your machine only**
>
> A commit is not a backup.